

09. BERT

Overview

BERT pre-trains bidirectional context with masked language modeling and next-sentence prediction, then fine-tunes on downstream tasks. This module explains tokenization with WordPiece, segment embeddings, the [CLS] pooling head, and fine-tuning hyperparameters for classification.

Lab: Lab 09.

Prior modules: Module 04.

Contents

1	Pre-training Paradigm	2
2	Model Architecture	2
3	Masked Language Modeling	2
4	Next Sentence Prediction	2
5	Fine-Tuning Recipe	2
6	Comparison with Seq2seq Encoders	2
7	Lab	3
8	Tokenization and Subwords	3
8.1	Whole-Word Masking	3
9	Extended Intuition	3
9.1	MLM Mechanics Beyond the Paper Diagram	3
10	Numerical Walkthrough	4
11	PyTorch Implementation Notes	4
12	Local GPU Constraints	5
13	Debugging Checklist	5
14	Practice Problems	5

Module track

This module builds on **Module 04**. Read those PDFs first. References to other modules appear as **Module NN** (not page numbers, because each module is its own PDF).

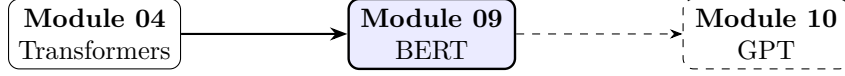


Figure 1: Module sequence (solid box: this PDF; dashed: typical next step).

1 Pre-training Paradigm

BERT [1] pre-trains deep bidirectional representations from unlabeled text, then fine-tunes on downstream tasks. Uses Transformer encoder blocks from **Module 04** (transformer block).

2 Model Architecture

BERT-base: $L = 12$, $d = 768$, $h = 12$, $\approx 110\text{M}$ parameters. Input representation sums token, segment, and position embeddings:

$$\mathbf{E}_{\text{input}} = \mathbf{E}_{\text{token}} + \mathbf{E}_{\text{segment}} + \mathbf{E}_{\text{position}}. \quad (1)$$

[CLS] token embedding aggregates sequence-level predictions.

Bidirectional context. Unlike causal LMs in **Module 10** (causal LM objective), BERT attends to full left and right context via masked language modeling.

3 Masked Language Modeling

Randomly mask 15% of tokens; of those, 80% replaced with [MASK], 10% random token, 10% unchanged.

$$\mathcal{L}_{\text{MLM}} = - \sum_{i \in \mathcal{M}} \log P(x_i | \tilde{\mathbf{x}}), \quad (2)$$

where $\mathcal{M}$ is the set of masked positions.

4 Next Sentence Prediction

Binary classification: is sentence B the actual next sentence after A ?

$$P(\text{IsNext} | A, B) = \text{softmax}(\mathbf{W}_{\text{NSP}} \mathbf{h}_{[\text{CLS}]}) . \quad (3)$$

Later work (RoBERTa) shows NSP is optional for many tasks.

5 Fine-Tuning Recipe

Add task head on [CLS] (classification) or token outputs (NER). Hyperparameters: lr 2×10^{-5} – 5×10^{-5} , batch 16 to 32, 2 to 4 epochs.

6 Comparison with Seq2seq Encoders

BERT encoder parallels bidirectional LSTM encoders in **Module 02** (bidirectional LSTM) but with full self-attention in **Module 04** (self-attention).

$$\text{F1}_{\text{QA}} = \frac{2 \cdot \text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}}. \quad (4)$$

Table 1: BERT fine-tuning heads.

Task	Head
Classification	Linear on [CLS]
Token labeling	Linear per token
QA	Span start/end logits

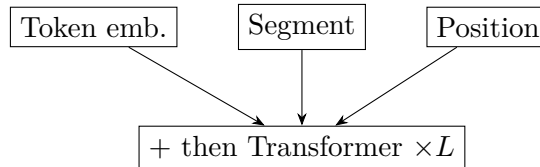


Figure 2: BERT input assembly.

7 Lab

Lab 09 fine-tunes BERT on sentiment; compare to GPT-style fine-tuning in **Module 10** (causal LM objective). Parameter-efficient tuning via LoRA in **Module 11** (LoRA definition).

8 Tokenization and Subwords

WordPiece/BPE splits rare words; vocabulary $\mathcal{V}$ typically 30k tokens. Max sequence length 512 limits quadratic attention cost from **Module 04**.

8.1 Whole-Word Masking

Mask entire word pieces belonging to one word, improves MLM coherence vs. random subword masks.

9 Extended Intuition

BERT pre-trains on masked language modeling (MLM): randomly mask 15% of tokens, predict them using bidirectional context. Unlike left-to-right LMs, BERT sees tokens to the left and right of a mask simultaneously through self-attention without causal masking. Next sentence prediction (NSP) was in the original paper; many later checkpoints drop it, but the idea was to learn document-level coherence.

Fine-tuning adds a task head on top of the [CLS] token representation for classification, or token heads for tagging. WordPiece tokenization splits rare words into subwords; the vocabulary (30k for **bert-base**) balances coverage and table size. Learning rates stay small (2×10^{-5} to 5×10^{-5}) because pretrained weights already encode syntax and semantics.

Compare with GPT-style causal models in **Module 10**: BERT excels at understanding tasks (classification, NER), GPT at generation. LoRA in **Module 11** reduces memory when fine-tuning large BERT variants on consumer GPUs.

9.1 MLM Mechanics Beyond the Paper Diagram

Of the 15% selected tokens, BERT replaces 80% with [MASK], 10% with random tokens, 10% unchanged. That trick forces representations that do not assume every input id is always visible at train time. Whole-word masking (later checkpoints) masks all WordPiece pieces of a word together, aligning MLM targets with linguistic units. At fine-tune time you never feed [MASK] tokens; only

pre-training uses them, so do not confuse MLM with fill-in-the-blank inference. ALBERT uses factorized embeddings and cross-layer parameter sharing; smaller footprint than **bert-base** when VRAM blocks full fine-tune before **Module 11**. NSP (next sentence prediction) pre-training is deprecated in RoBERTa-style pipelines; MLM alone drives most downstream gains on GLUE-scale tasks.

10 Numerical Walkthrough

bert-base-uncased: $L = 12$, $d = 768$, heads $H = 12$, FFN 3072, vocab $V = 30522$, max length $T = 128$, $B = 16$.

Token ids shape $(16, 128)$; embeddings sum token, segment, and position vectors $\in \mathbb{R}^{768}$. One self-attention layer: Q/K/V projections $768 \rightarrow 768$ each; attention $O(T^2 d) = 128^2 \cdot 768 \approx 12.6\text{M}$ multiply-adds per layer per sequence (rough order).

Fine-tune SST-2 binary sentiment: [CLS] vector $\mathbf{h}_{\text{CLS}} \in \mathbb{R}^{768}$ passed through **Linear(768, 2)**. Cross-entropy loss on batch; 2094 train examples, 2 epochs, $\text{lr} = 3 \times 10^{-5}$, warmup 10% steps. Expect val accuracy 90 to 93% with proper max-length padding and attention mask.

Trainable params full fine-tune: $\approx 110\text{M}$ (entire BERT + head). AdamW memory $\approx 3 \times$ params in fp32 $\rightarrow 1.3\text{GB}+$ weights alone; activations push total past 4GB at $B = 16$, $T = 128$.

Segment embeddings (**token_type_ids**) distinguish sentence A vs B in pair tasks; for single-sentence SST-2, set all zeros. Warmup-linear decay: first 10% of steps ramp lr from 0 to 3×10^{-5} , then linear decay to 0; jumping lr without warmup often spikes loss on step 1. MLM pre-training loss on masked positions only; fine-tune CE averages over batch labels, a much smaller gradient per token than MLM.

[CLS] token id is always position 0 after tokenization; pooling elsewhere grabs a punctuation embedding and tank accuracy silently. Max sequence length 128 truncates tail words in long reviews; for SST-2 most sentences fit, but log truncation rate on your split before trusting metrics. Whole-word masking groups WordPiece tokens; fine-tune does not use masking, but tokenizer still splits “running” into **run** + **##ning**, affecting rare-word generalization. Pair-classification (MNLI) needs two segments with **token_type_ids** 0/1; SST-2 single segment uses all-zero type ids.

11 PyTorch Implementation Notes

- Hugging Face sequence classifier:

```
BertForSequenceClassification.from_pretrained(
    "bert-base-uncased", num_labels=2)
```

- Pass **attention_mask** (1 for real tokens, 0 for pad) and **token_type_ids** if model expects them.

```
AutoTokenizer(..., truncation=True, padding="max_length", max_length=128)
```

For batch encoding.

- Freeze lower layers optionally: `for p in model.bert.encoder.layer[:6].parameters(): p.requires_grad = False.`
- Use **get_linear_schedule_with_warmup** with optimizer step count matched to dataloader $\text{length} \times \text{epochs}$.

```
outputs = model(input_ids, attention_mask=attention_mask, labels=labels)
loss = outputs.loss # includes classification head
loss.backward()
torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
```

12 Local GPU Constraints

bert-base fine-tune at $T = 128$, $B = 8$ fits 4 GB with fp32; use $B = 4$ if OOM during backward. **bert-large** is impractical full fine-tune on 1650; use **Module 11** LoRA or smaller distilled models.

Enable fp16 with `TrainingArguments(fp16=True)` in Hugging Face Trainer on Pascal+ GPUs. Gradient accumulation 4 with $B = 4$ gives effective batch 16 without linear memory growth.

DistilBERT retains 97% of **bert-base** quality at 40% size; swap in when 4 GB blocks full BERT fine-tune before trying LoRA. Attention head probing: some heads specialize in delimiter tokens ([SEP], punctuation); pruning heads rarely helps SST-2 but matters in structural tasks.

13 Debugging Checklist

- All predictions class 0: wrong label mapping or learning rate too high destroying pretrained weights.
- Loss NaN: fp16 overflow; try full fp32 first epoch or lower lr.
- Worse than linear baseline: max length too short truncating sentiment cues, or forgetting attention mask.
- Slow training on 1650: dataloader bottleneck; cache tokenized tensors to disk.
- Checkpoint load error: mismatched `num_labels` or wrong tokenizer vocabulary file.

CLS logits random while MLM head works: classification head not trained long enough or lr $10\times$ too high for head-only warmup phase.

14 Practice Problems

Problem 1. How many MLM labels per sequence of length 128 if 15% of tokens are masked (approx)?

Problem 2. Fine-tune with lr 10^{-4} vs 3×10^{-5} ; plot val accuracy vs epoch for both.

Problem 3. Freeze bottom 6 layers; compare trainable param count and peak VRAM to full fine-tune.

Problem 4. Extract [CLS] embeddings for 500 sentences and visualize with PCA colored by label.

Problem 5. Compare last-hidden-state mean pooling vs [CLS] pooling on SST-2 without changing the pretrained backbone.

Problem 6. Tokenize the same sentence with and without `clean_up_tokenization_spaces=False`. Confirm [CLS] position and length stay stable.

Dynamic padding (`padding=True` without fixed `max_length`) saves compute but requires passing `attention_mask` every forward pass. Evaluate with `model.eval()` and disable dropout; forgotten `eval()` on BERT drops val accuracy 2 to 5 points on small sets. Export `save_pretrained` after fine-tune includes config and tokenizer; loading with mismatched `num_labels` silently reshapes the classifier head. Run validation with identical `max_length` and truncation policy as training; eval-only longer context changes score without retraining. Distilled models (**distilbert-base-uncased**) trade 3 to 4 points accuracy for 40% faster inference on CPU baselines. Cache tokenized datasets to disk after first epoch; tokenization dominates wall time on SST-2 with naive Hugging Face loops. Report both accuracy and F1 on imbalanced dev sets; accuracy alone hides majority-class predictors.

Complete **Lab 09**; contrast with causal fine-tuning in **Module 10** and LoRA in **Module 11**.

References

- [1] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *NAACL*, 2019.
- [2] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *ICLR*, 2015.
- [3] Yinhan Liu, Myle Ott, Naman Goyal, et al. RoBERTa: A robustly optimized BERT pretraining approach. *arXiv preprint arXiv:1907.11692*, 2019.
- [4] Alec Radford, Jeffrey Wu, Rewon Child, et al. Language models are unsupervised multitask learners. *OpenAI Technical Report*, 2019.
- [5] Ashish Vaswani, Noam Shazeer, Niki Parmar, et al. Attention is all you need. In *Advances in Neural Information Processing Systems*, 2017.